

```
"""
```

Project 2: Basic Encryption & Decryption

```
-----
```

Implements a Caesar cipher (shift cipher) with:

- Encryption of user text
- Decryption of the encrypted text
- Display of both encrypted and decrypted output
- Edge case handling (spaces, punctuation, digits, upper/lower case)

Math (IPO Model):

Encrypt: $E_n(x) = (x + n) \% 26$

Decrypt: $D_n(x) = (x - n) \% 26$

where x = letter position (A=0..Z=25), n = shift key

```
"""
```

```
def encrypt(text: str, shift: int) -> str:
```

```
    """Encrypt plaintext using a Caesar cipher with the given shift."""
```

```
    result = []
```

```
    for char in text:
```

```
        if char.isupper():
```

```
            # A=65 .. Z=90
```

```
            shifted = (ord(char) - 65 + shift) % 26 + 65
```

```
            result.append(chr(shifted))
```

```
        elif char.islower():
```

```
            # a=97 .. z=122
```

```
            shifted = (ord(char) - 97 + shift) % 26 + 97
```

```
            result.append(chr(shifted))
```

```
        else:
```

```
            # Non-alphabetic characters (spaces, punctuation, digits) pass through
```

```
            result.append(char)
```

```
    return "".join(result)
```

```
def decrypt(cipher: str, shift: int) -> str:
```

```
    """Decrypt ciphertext using the same shift key (reverse shift)."""
```

```
    return encrypt(cipher, -shift)
```

```
def main():
```

```
    print("=== Caesar Cipher: Encryption & Decryption ===\n")
```

```
    plaintext = input("Enter text to encrypt: ")
```

```

# Validate the shift key
while True:
    try:
        shift = int(input("Enter shift key (integer, e.g. 3): "))
        break
    except ValueError:
        print("Please enter a valid whole number for the shift key.")

encrypted = encrypt(plaintext, shift)
decrypted = decrypt(encrypted, shift)

print("\n--- Results ---")
print(f"Plaintext : {plaintext}")
print(f"Shift Key : {shift}")
print(f"Encrypted : {encrypted}")
print(f"Decrypted : {decrypted}")

# Validation check
status = "PASSED" if decrypted == plaintext else "FAILED"
print(f"\nValidation (decrypt(encrypt(x)) == x): {status}")

if __name__ == "__main__":
    main()

```